

A FRAME VILLA CREATION ASSIGNMENT

DOCUMENTATION

REQUIREMENTS FOR ASSIGNMENT:

- At least three rooms (e.g., living room, bedroom, kitchen) with furniture and decorations.
- Lights, home decor, and animations implemented using the A-Frame entity-component system.
- An animated swimming pool with water in motion, implemented using the A-Frame physics system or a custom script.
- Additional models such as trees, plants, vehicles, or pets to enhance engagement.
- **The VR experience must be optimized for smooth performance, high-quality textures, and intuitive controls.**

ASSET ACQUISITION AND SCENE CONSTRUCTION

+ I acquired .max project of a House Exterior Model by Ngoc Quan was acquired, cleaned up in 3ds Max and exported as an FBX to be imported in Blender where the scene building and ideation was to happen.

+ I used blender for the layout/scene building because it's relatively easy to use, easy on memory and addons/plugins for modelling workflows where relatively cheaper and easier to get.

+ The House Exterior Model by Ngoc Quan was a house shell that had to be decorated.

+ I acquired some plant models, tree models and some other decorative ornaments together with textures from polyhaven.com – a trusted and free website that hosts high quality 3D assets.

+ I also acquired some car models and other design components from older projects and miscellaneous.

DESIGN CHOICES & CHALLENGES

- + It was only appropriate to have a two storey villa other than a flat building even though that would pose some navigation challenges in Aframe, yet some work arounds made it worthwhile.
- + Realism of the scene was a priority as such there had to be a LOD that was not so high but not so low instead, and acquiring different models of different format and different texture qualities made that a little challenging.
- + Most models had to be decimated and re-meshed just to get the face count to something usable for web – the project has to be highly optimized for work.
- + Care was taken to make sure the models didn't exceed 20mb during export; In that regard, models were exported as collections instead of as single scene.
- + The export format was .gltf instead of .glb because I needed the textures, .bin and the model file to be separate of each other to accommodate the less than 20 mb notion.
- + the already re-meshed and decimated models were also compressed further using Blender's Draco Compression for .gltf where the compression level was as high as 10 and quantization level was as high as 19. (Optimization was really key)
- + The textures had to be optimized as well. All and any texture worked on, be it a model's texture or a picture had to be converted from its original format to .webp and compressed to 33% of its original quality.
- + In the spirit of optimization for Aframe, some objects had their textures baked in blender to make sure there's realism at minimal cost in Aframe.
- + In order to get the sky texture that I wanted, a 2k .hdr file was converted into a 1k .jpg file that was used as the sky entity's src.
- + To make sure the pool was animated as to the requirements and also have the scene as optimized as possible, care was taken to avoid all Aframe physics libraries that would slow the scene; Instead an old trick in the book was employed where two normal textures are used and animated in opposite directions to sell the illusion of wave and the animation written in js as a script tag in the .html file.
- + Aframe's camera navigation is not very intuitive, to access the +/-y axes of the scene would have been tricky since Aframe doesn't normally have a very intuitive way of making that happen. To solve this, empties were created for every major collection, eg. Livingroom had `livingroom\_camnav` as an empty, pool area had `pool\_camnav` as an empty – `camnav` because I had to parent cameras to the empties to make sure the positions

were ideal after export. After setting the empties in the scenes, they were placed in a collection called `scene\_nav` where a python script:

```
import json, bpy

# Get the 'scene navs' collection

scene_navs = bpy.data.collections.get('scene navs')

if not scene_navs:

    raise Exception("No collection called 'scene navs' found")

# Filter empties in that collection

empties = [obj for obj in scene_navs.objects if obj.type == 'EMPTY']

# Export name → world position & rotation

data = {e.name: {

    'position': list(e.matrix_world.translation),

    'rotation': list(e.matrix_world.to_euler())

} for e in empties}

# Write JSON to specified path

json_path = r"C:\Users\hi_fr\Desktop\something\scene_navs.json"

with open(json_path, "w") as f:

    json.dump(data, f, indent=2)

print(f"Exported {len(empties)} empties to {json_path}")
```

written to target them would extract their blender world coordinates into a .json file into the same directory I was working in, so it would be manually used to edit the .html file or to write a .js script that spawns the camera to places Aframe camera would not let keyboard + mouse navigations get to.

The results were fed into buttons that Aframe would use to spawn the camera to certain locations.

+ The main challenge was not able to go all out because of the file size and scene optimization constraints.

+ To make sure the zip file was less than or equal to the 50mb threshold, aggressive compression tactics had to be used. For a insane compression .7z was opted as the best choice with compression level set to 9 – ultra, compression method set to LZMA2, an algorithm that makes me prefer .7z to .zip, the rest were just how much system resources I was willing to assign to the compression.

Overall the requirements were met to the best of my abilities.